Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Monterrey

Modelación del Aprendizaje con Inteligencia Artificial
TC2034 (Grp 201)

Nombre del profesor:
Hugo Terashima Marín

Proyecto 1:
Implementación y Análisis Comparativo de Modelos de Regresión mediante Técnicas de Optimización por Descenso de Gradiente

Laurie Camila Hernández Pacheco — A01286569
Fernando Álvarez Ruiz — A01286754

Fecha: 15 de abril, 2026

Implementación y Análisis Comparativo de Modelos de Regresión mediante Técnicas de Optimización por Descenso de Gradiente

Fernando Álvarez Ruiz y
Laurie Camila Hernández Pacheco

Instituto Tecnológico y de Estudios Superiores de Monterrey y
Profesor: Hugo Terashima Marín
(Dated: 16 de abril de 2026)

ABSTRACT: En este trabajo se implementaron desde cero los modelos de regresión lineal y regresión logística utilizando el algoritmo de descenso del gradiente como método de optimización. El objetivo principal fue comprender a profundidad los fundamentos matemáticos de estos modelos y evaluar su desempeño en comparación con las implementaciones de la librería *scikit-learn*. Para ello, se emplearon dos conjuntos de datos: California Housing Prices para regresión lineal y Telco Customer Churn para regresión logística, aplicando técnicas de preprocesamiento como estandarización y codificación de variables.

Los resultados obtenidos muestran que la implementación manual es capaz de reproducir con alta precisión el comportamiento de los modelos estándar. En regresión lineal, se logró una coincidencia exacta con la solución analítica al utilizar valores adecuados de la tasa de aprendizaje, evidenciando la convergencia del descenso del gradiente hacia el mínimo global. En regresión logística, aunque se observaron ligeras diferencias en métricas como precisión, recall y AUC-ROC, el modelo manual alcanzó un desempeño altamente comparable al de *scikit-learn*.

En conjunto, este estudio demuestra que una correcta implementación de los fundamentos matemáticos, acompañada de un adecuado preprocesamiento y ajuste de hiperparámetros, permite replicar eficazmente modelos clásicos de aprendizaje automático, fortaleciendo la comprensión teórica y práctica de estos algoritmos.

Palabras Clave: Aprendizaje Automático, Regresión Lineal, Regresión Logística, Descenso del Gradiente, Optimización, Preprocesamiento de Datos, scikit-learn, Modelos Supervisados

CONTENTS

I. Introducción 2

II. Planteamiento del Problema 2

III. Objetivo del trabajo 2

IV. Metodología 2
- A. Descenso del Gradiente 2
  - Implementación en regresión lineal 3
  - Implementación en regresión logística 3
- B. Modelo Lineal 3
  - Exploración de Datos 3
  - Implementación Manual 4
  - Entrenamiento del Modelo 4
  - Evaluación del Modelo 4
- C. Modelo Logístico 4
  - Exploración de Datos 5
  - Implementación Manual 5
  - Entrenamiento del Modelo 5
  - Evaluación del Modelo 5
- D. Experimentos realizados 6
  - Regresión Lineal: Pendiente e Intersección 6
  - Regresión Logística: Curvas ROC 7

V. Resultados y Discusión 8
- A. Regresión Lineal 8
- B. Regresión Logística 9

VI. Conclusión 10

Referencias 11

I. INTRODUCCIÓN

El aprendizaje automático, o *machine learning* (ML por sus siglas en inglés), es un subconjunto de la inteligencia artificial (IA) que estudia cómo las computadoras pueden mejorar su desempeño a partir de la experiencia [1]. A pesar de ser un área relativamente reciente, su crecimiento ha sido acelerado, impulsando el desarrollo de nuevas formulaciones de problemas y una amplia variedad de aplicaciones prácticas. Entre estas se encuentran la predicción de enfermedades a partir de datos clínicos, la detección de volcanes activos en otros planetas mediante el análisis de imágenes, y los sistemas de recomendación de contenido en redes sociales [2].

Dentro del ML, existen tres paradigmas principales: aprendizaje supervisado, no supervisado y por refuerzo [1]. Cada uno de estos enfoques permite abordar distintos tipos de problemas dependiendo de la disponibilidad y naturaleza de los datos. En particular, el aprendizaje supervisado resulta fundamental en tareas de predicción y clasificación, donde se cuenta con datos etiquetados que permiten entrenar modelos capaces de generalizar patrones y realizar inferencias sobre nueva información.

En este proyecto se exploran técnicas de aprendizaje supervisado mediante la implementación de dos modelos fundamentales: la regresión lineal y la regresión logística. Ambos modelos se entrenan utilizando el algoritmo de descenso de gradiente: una forma de minimizar la función objetivo parametrizada $J(\theta)$ a través de sus parámetros $\theta \in \mathbb{R}^d$ actualizándolos en dirección en contra del gradiente de la función objetivo $\nabla_\theta J(\theta)$ [3]. Para comprender su funcionamiento y diferencias en su proceso de optimización, se desarrollarán implementaciones propias de cada modelo y se comparan sus resultados con aquellos obtenidos mediante la librería *scikit-learn*. A lo largo del documento se describe el proceso de implementación, el entrenamiento de los modelos, la evaluación de su desempeño y el análisis comparativo de los resultados obtenidos.

II. PLANTEAMIENTO DEL PROBLEMA

Dentro del aprendizaje supervisado, el objetivo principal es inferir una función $f : X \to Y$, también conocida como clasificador, a partir de información de muestra A_n compuesta de parejas de puntos (x_n, y_n) donde $x_i \in X$ y $y_i \in Y$. [4]

Si bien ya existen herramientas altamente optimizadas, el uso de estas a veces oculta los detalles fundamentales de los algoritmos. En los modelos de regresión lineal y regresión logistica se construyen las bases esenciales del aprendizaje automático, siendo ampliamente utilizados en problemas reales gracias a su interpretabilidad y eficiencia.

Por lo tanto, surge la necesidad de implementar estos modelos desde cero, con el fin de estudiar de manera detallada su funcionamiento, identificar sus limitaciones y comprender el impacto de distintos factores en su desempeño. Asimismo, resulta relevante comparar dichas implementaciones con herramientas consolidadas como *scikit-learn*, lo que permite validar resultados y analizar diferencias en términos de precisión y eficiencia. Este enfoque no solo fortalece la comprensión teórica, sino que también desarrolla habilidades prácticas en la construcción de modelos de aprendizaje automático.

III. OBJETIVO DEL TRABAJO

Este proyecto tiene como objetivo implementar desde cero los modelos de regresión lineal y logística a través del algoritmo de descenso de gradiente como método de optimización. Se busca entrenar estos modelos usando conjuntos de datos predefinidos, evaluar su desempeño mediante métricas apropiadas y finalmente comparar los resultados obtenidos con aquellos generados por las implementaciones de la librería *scikit-learn*.

IV. METODOLOGÍA

Para este trabajo, la metodología en cada uno de los modelos es similar. No obstante, vale la pena considerar que para cada modelo se sigue un procedimiento diferente. Para ambos trabajos, inicialmente se planteó un entorno en un archivo de Jupyter Notebook utilizando Python 3.12, donde se importaron las siguientes librerías:

- `pandas`: para manipulación de los datasets.
- `numpy`: para cálculos numéricos.
- `matplotlib`: para visualización y gráficos.
- `seaborn`: para visualización y mapas de calor.
- `sklearn`‘: para implementar los modelos de regresión lineal y logística.

A continuación se detalla la metodología para el modelo de regresión lineal y el de regresión logística respectivamente, así como el motor de funcionamiento de ambos modelos: el descenso del gradiente.

A. Descenso del Gradiente

El descenso del gradiente es un método de optimización utilizado para minimizar una función objetivo $J(\theta)$ mediante la actualización iterativa de sus parámetros en dirección opuesta al gradiente, lo que corresponde a la dirección de mayor descenso. En este trabajo se emplea

la variante de *batch gradient descent* (BGD), en la cual el gradiente se calcula utilizando la totalidad de las muestras del conjunto de entrenamiento en cada iteración [5].

La regla general de actualización de parámetros está dada por:

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla_\theta J(\theta) \tag{1}$$

donde α representa la tasa de aprendizaje del modelo.

Implementación en regresión lineal

Para la regresión lineal, los parámetros θ se descomponen en un vector de pesos $\mathbf{w}$ y un sesgo b, los cuales se inicializan en cero. En cada iteración, las predicciones del modelo se calculan como:

$$\hat{y} = X\mathbf{w} + b \tag{2}$$

El gradiente de la función de costo respecto a los parámetros se obtiene utilizando todas las muestras:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{n} X^T(\hat{y} - y), \quad \frac{\partial J}{\partial b} = \frac{1}{n} \sum (\hat{y} - y) \tag{3}$$

Con estos gradientes, los parámetros se actualizan iterativamente hasta aproximarse a un mínimo de la función de costo:

$$\mathbf{w} = \mathbf{w} - \alpha \frac{\partial J}{\partial \mathbf{w}}, \quad b = b - \alpha \frac{\partial J}{\partial b} \tag{4}$$

Implementación en regresión logística

En el caso de la regresión logística binaria, se mantiene el mismo esquema de optimización mediante BGD; sin embargo, se modifica la función de hipótesis y la función de costo para adaptarse a un problema de clasificación.

En lugar de predecir valores continuos, el modelo estima probabilidades mediante la función sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{donde } z = X\mathbf{w} + b \tag{5}$$

Por lo tanto, las predicciones se definen como:

$$\hat{y} = \sigma(X\mathbf{w} + b) \tag{6}$$

Para este modelo se utiliza la función de costo logarítmica (*log-loss*), adecuada para clasificación binaria. A partir de esta, los gradientes respecto a los parámetros toman la forma:

$$\frac{\partial J}{\partial \mathbf{w}} = \frac{1}{n} X^T(\hat{y} - y), \quad \frac{\partial J}{\partial b} = \frac{1}{n} \sum (\hat{y} - y) \tag{7}$$

A pesar del cambio en la función de costo, la estructura de actualización de parámetros se mantiene:

$$\mathbf{w} = \mathbf{w} - \alpha \frac{\partial J}{\partial \mathbf{w}}, \quad b = b - \alpha \frac{\partial J}{\partial b} \tag{8}$$

De esta manera, el modelo ajusta iterativamente sus parámetros utilizando la totalidad de los datos en cada paso, conservando el enfoque de BGD. [6]

Finalmente, para ambos modelos se implementa una función `fit`, encargada de ejecutar el proceso iterativo de entrenamiento, incluyendo el cálculo de predicciones, evaluación del gradiente y actualización de parámetros.

B. Modelo Lineal

Inicialmente, se realizó una búsqueda exhaustiva para hallar una base de datos relevante y válida para el trabajo. Se decidió trabajar con la base de datos *California Housing Prices*, un dataset obtenido de Kaggle que contiene información de un censo realizado en California durante 1990. Este dataset contiene las siguientes variables:

Variable	**Descripción**	**Tipo**
longitude	Coordenada oeste de Greenwich.	Float
latitude	Coordenada norte del ecuador.	Float
housing_median_age	Edad mediana de viviendas.	Float
total_rooms	Total de habitaciones del bloque.	Float
total_bedrooms	Total de dormitorios del bloque.	Float
population	Personas que viven en el bloque.	Float
households	Número de hogares.	Float
median_income	Ingreso mediano (10 mil USD).	Float
median_house_value	Valor mediano de viviendas.	Float
ocean_proximity	Cercanía al océano (5 categorías).	Categórica

Tabla I: Descripción de variables del conjunto de datos

Exploración de Datos

Inicialmente, se hace un análisis exploratorio de datos (EDA) donde se identificaron las dimensiones del dataframe y se realizó una descripción estadística de los datos, exceptuando la columna de `ocean_proximity`, pues es categórica. Se identificaron un total de 20640 filas y 10 columnas, donde la columna de `total_bedrooms` únicamente se tenían 20433 datos que no eran nulos. A partir de esto, se obtuvieron las cantidades de datos duplicados y nulos, cuyo resultado fue positivo, pues únicamente había 207 filas con datos nulos exclusivamente de la columna `total_bedrooms`. Por lo tanto, se tomó la decisión de eliminar esas filas, manteniendo al

final el 98,99 % de los datos totales.

Finalmente, por cuestiones de escala fue necesario modificar el orden de magnitud de la columna `median_income`, pues se encontraban los datos en decenas de miles de dólares, mientras que las demás columnas relacionadas a dólares se encontraban directas. Por lo tanto, se realizó la multiplicación adecuada para esa columna.

Ya preparado y modificado el dataset, se eligen las variables dependientes e independientes para la regresión lineal. En este caso, de variable independiente X se utiliza `median_income` y de variable dependiente y se utiliza `median_house_value`. El objetivo de este modelo de regresión lineal es hacer uso del ingreso mediano de cada cuadra para predecir el valor mediano de las casas en la misma cuadra. A partir de aquí, se implementaron los modelos manuales y de `scikit learn` para poder identificar diferencias entre ambos.

Implementación Manual

Para la implementación manual del modelo, se crea una clase llamada `RegresionLineal()`, que contiene su función inicializadora con los parámetros `lr` (tasa de aprendiaje α) y `n_iter` (número de iteraciones). Además se definen las variables `weights` y `bias`. Después, se presenta la función de `fit` que tiene de parámetros de entrada X, que es la matriz de valores de la variable independiente, y y, los valores reales de lo que se quiere predecir. En esta función se realizan las iteraciones adecuadas para ir ajustando los valores de `weights` y `bias`, actualizando de manera adecuada. Finalmente, se incluye la función `predict` que tiene como parámetro de entrada a X y realiza el cálculo para la predicción.

Entrenamiento del Modelo

Posteriormente, se procedió a realizar el entrenamiento del modelo con la implementación tanto manual como con `scikit learn`, dividiendo la base de datos entre datos de entrenamiento y prueba con una proporción 80 : 20. Antes de entrenar los datos, se aplica `StandardScaler` a ambas variables X e y. Esto es necesario porque el modelo manual usa GD, y dado que las variables tienen escalas muy diferentes, el algoritmo tarda mucho más en converger o no logra llegar al mínimo. El scaler permite transformar los datos de tal forma que tengan media de 0 y una desviación estándar de 1, lo que hace el entrenamiento más estable.

Sin embargo, se debe de considerar que el ajuste de la transformación se hace únicamente en los datos de entrenamiento, mientras que en los datos de prueba se aplica directamente. Esto evita que ocurran casos de *data leakage* entre los dos conjuntos de datos.

Finalmente se entrenan ambos modelos con los datos escalados: LinearRegression de *sklearn* (que usa solución analítica exacta) y la implementación manual RegresionLineal (que usa GD con lr=0.1 y 1000 iteraciones), y se generan las predicciones sobre el test set.

Evaluación del Modelo

Para evaluar el modelo se hace uso de las siguientes métricas:

- Mean Absolute Error (MAE): el promedio del valor absoluto de los errores, se calcula:

$$MAE = \frac{1}{n}\sum_{j=1}^{n}|y_j - \hat{y}_j|$$

- Mean Squared Error (MSE): el promedio de los errores al cuadrado, se calcula:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Root Mean Squared Error (RMSE): la raíz cuadrada del promedio de los errores al cuadrado, se calcula:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Existen funciones ya predeterminadas en la librería de `sklearn` para evaluarlos, por lo cual estas métricas serán utilizadas. Finalmente, también se realiza una comparación entre la pendiente e intersección con el eje y, y se incluye una visualización de los datos con respecto a las rectas de regresión logística generadas por ambos modelos.

C. Modelo Logístico

Inicialmente, se realizó una búsqueda para encontrar una base de datos que permitiera el manejo de variables binarias y se eligió el dataset *Telco Customer Churn*, el cual contiene un total de 21 variables. Después de eliminar variables que no parecen agregar información y convertir a binaria todas las categóricas, se modificaron a las que se muestran en la siguiente tabla:

Variable	Descripción	Tipo
customerID	ID del cliente	Categórica
gender	Género del cliente	Categórica
SeniorCitizen	Tercera edad (1 = sí, 0 = no)	Binaria
Partner	Tiene pareja o matrimonio	Categórica
Dependents	Tiene dependientes financieros	Categórica
Tenure	Meses continuos con el servicio	Numérica
PhoneService	Cuenta con servicio telefónico	Categórica
MultipleLines	Cantidad de líneas telefónicas	Categórica
InternetService	Tipo de conexión a internet	Categórica
OnlineSecurity	Seguridad en línea	Categórica
OnlineBackup	Respaldo en la nube	Categórica
DeviceProtection	Protección del dispositivo	Categórica
TechSupport	Soporte técnico	Categórica
StreamingTV	Servicio de TV en streaming	Categórica
StreamingMovies	Servicio de películas en streaming	Categórica
Contract	Tipo o frecuencia de contrato	Categórica
PaperlessBilling	Facturación electrónica	Categórica
PaymentMethod	Método de pago	Categórica
MonthlyCharges	Costo mensual del plan	Numérica
TotalCharges	Costo total acumulado	Numérica
Churn	Abandono del servicio	Binaria

Tabla II: Descripción de variables para la regresión logística

Si bien la mayoría de las variables son categóricas, una gran cantidad de ellas como `Churn` se clasifican en preguntas de tipo sí/no, por lo que es más sencillo convertirlas a variables numéricas.

Exploración de Datos

Se empieza con un análisis exploratorio de datos (EDA) de manera similar al modelo de regresión lineal. Inicialmente se consiguieron las dimensiones del dataset y su descripción estadística para las 3 variables en las que fue posible. Se identifican 7043 filas y 21 columnas distintas. Se convirtió primero la variable `total_charges` a numérica. En la descripción estadística, se nota que después de la conversión tiene 7032 observaciones únicamente, por lo cual se pueden identificar datos nulos en el dataset. Se revisan después el total de valores nulos, obteniendo un total de 11 valores, todos provenientes de la columna `total_charges`. Debido a que únicamente forman el 0,156 % del total de datos, son eliminados.

Parte del procesamiento de los datos requiere convertir aquellas columnas de categorías binarias a variables numéricas (partner, dependents, PhoneService y PaperlessBilling), representando Sí $\rightarrow$ 1 y No $\rightarrow$ 0. Se realiza el mapeo de esto para las variables necesarias. Asimismo, se eliminarán las categorías que no aportan información necesaria del modelo (`customerID` y `gender`) y se procesarán únicamente las columnas necesarias. Para ello, se utilizará nuevamente el `StandardScaler`, así como también `One-Hot Encoding` para codificar las categorías múltiples. También se separará la variable objetivo `Churn` de los demás datos.

Implementación Manual

La regresión logística es un algoritmo de clasificación que permite catalogar resultados. En este caso, se enfoca específicamente en la regresión logística binomial, que permite catalogar datos en dos posibles categorías. A diferencia de la regresión lineal, no se busca un valor en específico, sino una probabilidad de qué tan posible es que el dato sea sí (1) o no (0). Esto se puede hacer a través de la función sigmoide (5), la cual se implementó manualmente en Python llamándose `sigmoid`.

Se creó una clase llamada `RegresionLogistica` que contiene la función de inicialización donde se reciben los parámetros de `lr` (tasa de aprendizaje α) y `n_iter` (número de iteraciones), y también se instancían ambas `weights` y `bias` en 0. Después, se presenta la función `fit`, que recibe similar a la función de la clase `RegresionLineal` a las variables X e y y lleva a cabo el proceso completo de iteraciones para ajustar los parámetros. Finalmente, se cuenta con la función `predict` que recibe igualmente X y lleva a cabo el proceso de cálculo de predicción de los datos, considerando también el uso de la función sigmoide.

Entrenamiento del Modelo

Para el entrenamiento del modelo, se dividieron los datos en los conjuntos de entrenamiento y prueba con una proporcion de 75:25. Posteriormente, se probó con ambos modelos manuales y de `scikit learn` de tal forma que se puedan comparar los mismos resultados. Para el modelo manual, se utilizaron los parámetros $\alpha = 0{,}01$ y n_iter = 10000 con el objetivo de apoyar al modelo manual a llegar al óptimo.

Evaluación del Modelo

Finalmente, es necesario evaluar el rendimiento de cada uno de los modelos de regresión logística, lo cual se lleva a cabo utilizando las siguientes herramientas:

- Matriz de confusión: permite visualizar datos como los falsos positivos y falsos negativos, así como también aquellas predicciones correctas e incorrectas que realizó el modelo. A partir de esta matriz se pueden identificar demás métricas como la precisión, recall, f1-score y la exactitud del modelo.

- Curva ROC: gráfica que muestra la relación entre la tasa de verdaderos positivos (recall) y la tasa de falsos positivos para distintos valores de threshold del modelo.

Estas métricas se calculan para ambos modelos manuales y el realizado por `scikit learn` con el objetivo de visualizar diferencias en los mecanismos de predicción.

D. Experimentos realizados

Con el objetivo de analizar el comportamiento de los modelos implementados, se llevaron a cabo diversos experimentos enfocados en la sensibilidad a hiperparámetros y la convergencia del algoritmo de descenso del gradiente.

En particular, se evaluó el impacto de la tasa de aprendizaje α y el número de iteraciones sobre el proceso de entrenamiento. Se probaron distintos valores de α con el fin de identificar escenarios donde el modelo converge de manera adecuada, así como casos donde una tasa demasiado alta genera inestabilidad o divergencia, y valores demasiado bajos provocan una convergencia lenta.

Asimismo, se monitoreó la evolución del error a lo largo de las iteraciones para verificar el comportamiento de convergencia del modelo. Esto permitió validar la correcta implementación del algoritmo de descenso del gradiente y detectar la necesidad de aplicar técnicas de preprocesamiento, como la estandarización de los datos mediante `StandardScaler`, con el fin de mejorar la estabilidad numérica del entrenamiento.

Adicionalmente, se realizaron comparaciones entre la implementación manual y los modelos proporcionados por `scikit learn`, analizando diferencias en desempeño y precisión. Para ello, se utilizaron las métricas definidas previamente (MAE, MSE y RMSE para regresión lineal, así como métricas de clasificación para regresión logística).

Finalmente, los experimentos permitieron identificar configuraciones de hiperparámetros que logran un balance adecuado entre precisión del modelo y costo computacional, evitando un número excesivo de iteraciones o valores de α que comprometan la estabilidad del entrenamiento.

Regresión Lineal: Pendiente e Intersección

Para la regresión lineal, se realizó experimentos alternando el valor de α para el modelo manual y visualizando su ajuste con aquel de `scikit learn`. A continuación se muestran las gráficas para tres casos: $\alpha = 0{,}001$, $\alpha = 0{,}01$ y $\alpha = 0{,}1$.

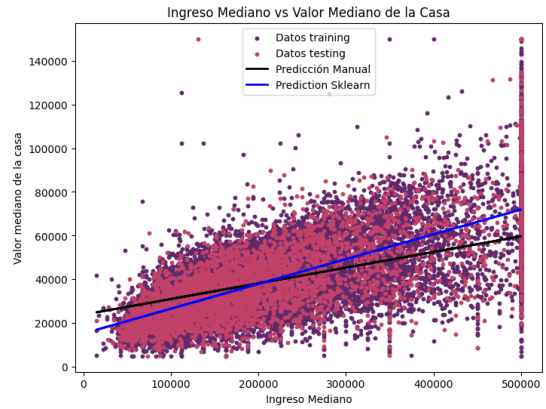

Figura 1: Recta de regresión lineal para $\alpha = 0{,}001$. Autoría propia

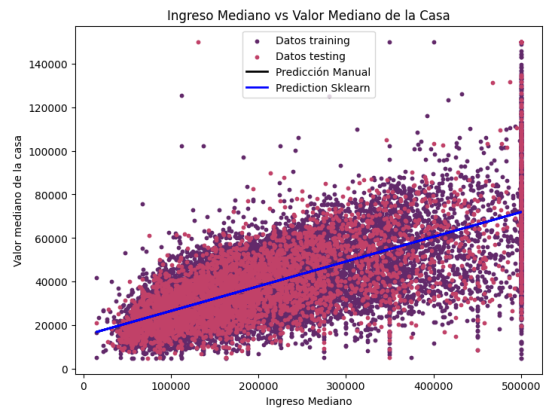

Figura 2: Recta de regresión lineal para $\alpha = 0{,}01$. Autoría propia

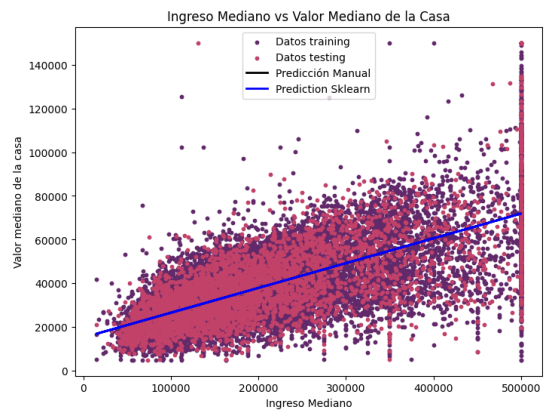

Figura 3: Recta de regresión lineal para $\alpha = 0{,}1$. Autoría propia

Se puede visualizar cómo en la figura 1 (la recta con la menor α, la línea se encuentra con una pendiente menor a la recta presentada por el modelo de `scikit learn`.

Mientras tanto, para ambas $\alpha = 0{,}01$ y $\alpha = 0{,}1$ se muestra un mucho mejor ajuste, donde la línea que representa al modelo manual es casi despreciable. Sin embargo, las diferencias son más significativas cuando se comparan los errores entre las rectas:

Métrica	**Sklearn**	**Manual**
R^2	0.4776	0.4137
MAE	10093.5785	10801.7873
RMSE	13670.8633	14482.9403
MSE	186892502.5900	209755559.2629

Tabla III: Resultados del Caso 1: $\alpha = 0{,}001$. Autoría propia.

Métrica	**Sklearn**	**Manual**
R^2	0.4776	0.4776
MAE	10093.5785	10093.5943
RMSE	13670.8633	13670.8616
MSE	186892502.5900	186892457.2550

Tabla IV: Resultados del Caso 2: $\alpha = 0{,}01$. Autoría propia.

Métrica	**Sklearn**	**Manual**
R^2	0.4776	0.4776
MAE	10093.5785	10093.5785
RMSE	13670.8633	13670.8633
MSE	186892502.5900	186892502.5900

Tabla V: Resultados del Caso 3: $\alpha = 0{,}1$. Autoría propia.

La diferencia de los errores en los primeros casos es notable entre el modelo de `sklearn` y el manual, pues se presentan mayores discrepancias. Sin embargo, conforme aumenta α, disminuyen estos errores eventualmente convergiendo al mismo (o casi el mismo) punto óptimó.

Regresión Logística: Curvas ROC

Para la regresión logística, se experimenta con las curvas ROC y el aumento del área bajo su curva

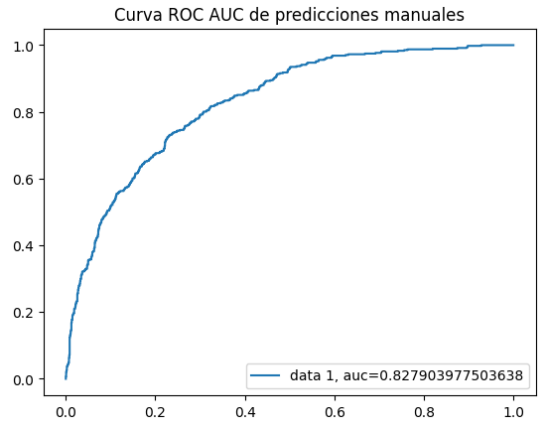

Figura 4: Curva ROC para $\alpha = 0{,}001$ Autoría propia

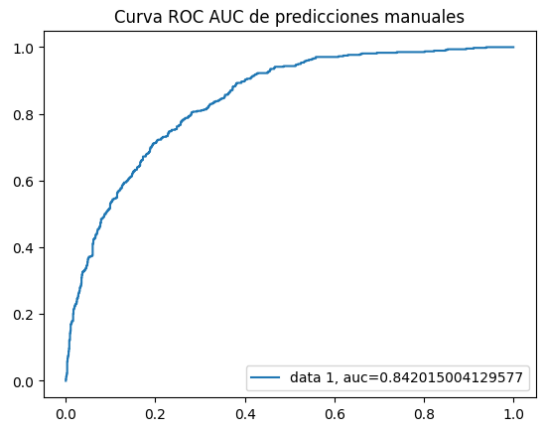

Figura 5: Curva ROC para $\alpha = 0{,}01$ Autoría propia

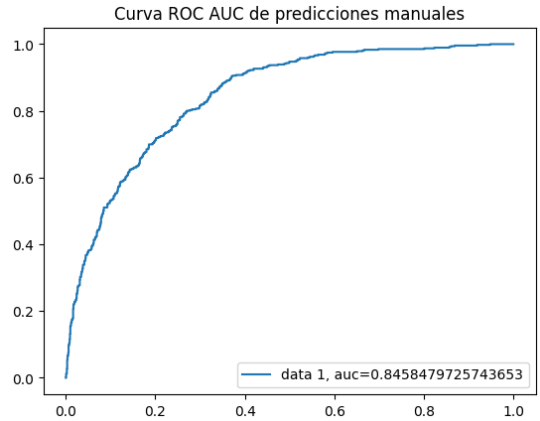

Figura 6: Curva ROC para $\alpha = 0{,}1$ Autoría propia

Se puede identificar cómo el área bajo la curva aumenta con respecto al aumento del hiperparámetro α. Se nota un aumento altamente significativo entre $\alpha = 0{,}001$ y $\alpha = 0{,}01$, pues aumenta 0.02 aproximadamente. Sin embargo, cuando se realiza el salto de $\alpha = 0{,}01$ a $\alpha = 0{,}1$ el cambio

si bien es positivo, no representa un gran impacto en términos del contexto así como tampoco del área bajo la curva, por lo que se decidió evitar su uso pues resulta poco eficiente.

V. RESULTADOS Y DISCUSIÓN

Ahora que ambas clases manuales fueron diseñadas y probadas, se pueden revisar sus diferencias con la ejecución directa de scikit-learn, que varían según el modelo a observar. Esto, con el fin de elaborar posteriormente una comparación entre ambos modelos.

A. Regresión Lineal

Se empezará con los resultados obtenidos de la regresión lineal implementada manualmente para encontrar la relación entre dos variables. El objetivo de esta regresión fue el de aplicar un modelo matemático iterativo para entender la manera en la que los precios de las casas de un bloque residencial en California está relacionado con los ingresos de las familias que viven ahí. Lo que se hizo con datos de la mediana de los ingresos y los precios de las casas de cada bloque. La base de datos también incluye más información sobre las casas, pero solo se tomarán estas dos variables porque solo se busca la relación entre estas.

Para mejorar la estabilidad de la implementación manual del descenso de gradiente, se escaló y estandarizó la información de ambas variables. Posteriormente, los datos fueron regresados a su valor original con el fin de mostrar la verdadera relación en término de las unidades reales (USD).

La comparación entre la información obtenida de la clase de Scikit y el modelo manual con descenso de gradiente se realizó por medio de los valores obtenidos para pendiente e intercepto, que son los que definen la línea. Además, se compararán las diferencias de los errores de ambas líneas de menor ajuste con los coeficientes R-cuadrada, Error medio absoluto (MAE), error cuadrático absoluto (MSE) y la raíz del MSE.

Los resultados obtenidos para la línea de menor ajuste fueron los siguientes:

Variable	Scikit-learn	Manual
Pendiente	0.1133	0.1133
Intercepto	15245.9934	15245.9934

Tabla VI: Comparación entre recta de ajuste manual y Scikit-learn

Si se grafican ambas rectas de ajuste junto con todos los puntos de datos, se puede visualizar de la siguiente manera:

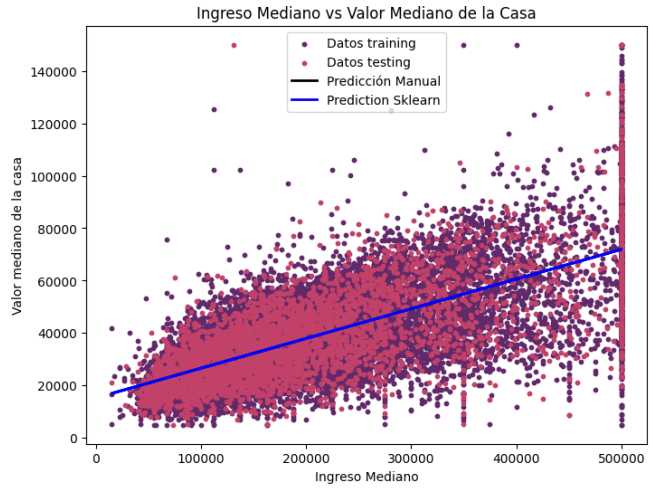

Figura 7: Visualización de gráfica de regresión lineal con ambas rectas manual y la de scikit learn. Autoría propia

Finalmente, se presentan los distintos tipos de errores entre ambas rectas de ajuste y los puntos de datos obtenidos en la realidad:

Variable	Scikit-learn	Manual
R^2	0.4776	0.4776
MAE	10093.5785	10093.5785
RMSE	13670.8633	13670.8633
MSE	186892502.5900	186892502.5900

Tabla VII: Comparación entre tipos de errores de regresión manual y Scikit-learn

Los resultados muestran una coincidencia completa de los valores tanto de la recta de mejor ajuste, como de todos los tipos de errores para ambos métodos de regresión. Esto indica que la reproducción manual del método con descenso de gradiente pudo replicar la solución exacta generada previamente con una librería especializada.

El resultado obtenido del coeficiente de determinación indica que un 47.76 % de la varianza de la variable dependiente se puede explicar por la independiente, lo que quiere decir que estas variables sí tienen una correlación decente entre ellas. No obstante, la prioridad de esta práctica fue el de observar la capacidad de simulación del método de la librería por medio de las iteraciones manuales, lo que fue logrado de la manera perfecta.

De esta manera, se puede evidenciar que la implementación manual fue consistente con todos los valores del modelo estándar de Scikit, consiguiendo llegar a la solución óptima. Este comportamiento va acorde a la

naturaleza matemática de la situación, ya que la función que se minimiza en la regresión lineal (MSE) es convexa y debe, por lo tanto, tener un solo mínimo (el global). Esto implica que el mínimo puede ser alcanzado por una función típica de descenso de gradiente en lugar de los métodos numéricos avanzados que se tienen que utilizar para otras funciones.

Los resultados de la implementación manual demuestran el uso correcto de las funciones matemáticas como el MSE, el descenso de gradiente y la técnica de repetición de iteraciones en el desarrollo de esta práctica. Además, enseñan que con los modelos matemáticos correctos y un buen preprocesamiento, se pueden simular modelos clásicos de machine learning sin tener que depender de librerías externas.

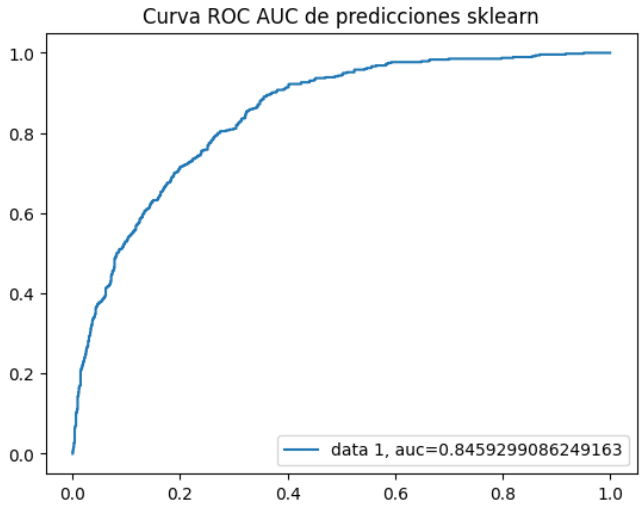

Figura 8: Curva ROC de predicción utilizando `scikit learn`. Autoría propia

B. Regresión Logística

Ahora se continuará con los resultados obtenidos de la implementación manual de la regresión logística. El objetivo de esta regresión fue enseñar al modelo a distinguir si un cliente se convierte en churner o si continúa renovando el servicio de la compañía, lo que se realizó por medio de la información de una base con datos de cada cliente escalada y codificada con técnicas de preprocesamiento pertinentes para la clasificación binaria.

Para evaluar la capacidad del modelo manual, se compararó con el resultado obtenido por el modelo estándar *LogisticRegression()* de la librería Scikit-learn, lo que hizo por medio de las métricas estándar de evaluación para la regresión logística binaria: precisión, recall, f1-score y el ROC-AUC. Los resultados (weighted) se muestran en la siguiente tabla:

Métrica	**Scikit-learn**	**Manual**
Precision	0.7898	0.7873
Recall	0.7964	0.7952
F1-score	0.7922	0.7898

Tabla VIII: Comparación entre regresión logística manual y Scikit-learn

Mientras tanto, los gráficos de las métricas ROC AUC son las siguientes:

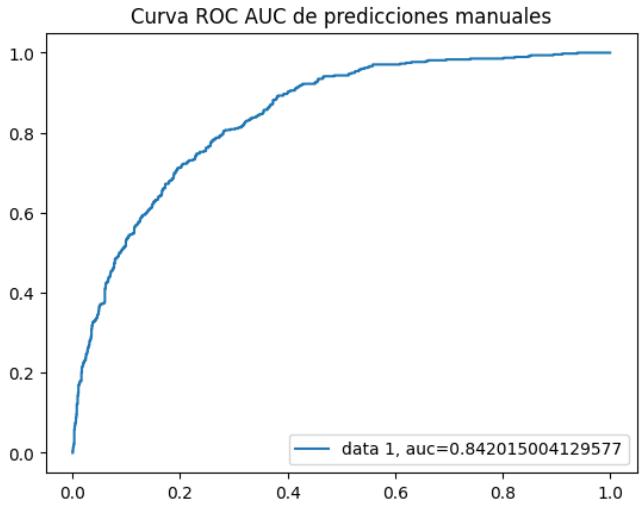

Figura 9: Curva ROC de predicción utilizando el modelo manual. Autoría propia

Ahora se compararán las matrices de confusión para entender la capacidad de ambos modelos de clasificar a los churners según el objetivo de la compañía, lo que se ve en las siguientes tablas:

	Predicho No	**Predicho Sí**
No	1131	151
Sí	207	269

Tabla IX: Matriz de confusión del modelo Scikit-learn

	Predicho No	Predicho Sí
No	1137	145
Sí	215	261

Tabla X: Matriz de confusión del modelo manual

Como se puede observar en la matriz de confusión, los resultados obtenidos por el modelo manual presentan una similitud muy elevada al compararse con el modelo de la librería Scikit. Por el lado de la clase negativa, es notable la diferencia de 8 clientes no-churners siendo reconocidos correctamente como no-churners (verdaderos negativos) por el modelo manual, lo que implica que este modelo es incluso mejor que el de la librería para detectar los casos negativos.

Por otro lado, también se presenta una diferencia en los resultados de la clase positiva. En la matriz de confusión del modelo manual existe una diferencia de 8 personas que pasan de estar clasificadas correctamente como churners en la tabla original, a estar incorrectamente clasificadas en la nueva tabla (falsos negativos), mostrando una ligera falla en la efectividad del modelo.

El hecho de que las diferencias en los errores de falsos negativos sean mínimamente mayores que los nuevos aciertos de verdaderos negativos es lo que provoca que la precisión, el recall y el f1-score del modelo manual resulten menores por un margen de una o dos décimas de porcentaje a su contraparte estándar, así como provocan que el valor AUC ROC sea también 3 milésimas más pequeño

Juntando estos resultados, se puede interpretar por las diferencias mínimas de la matriz y las métricas que el algoritmo manual alcanzó niveles de clasificación extremadamente cercanos a la versión original diseñada para la librería, con fronteras de decisión muy parecidas. Si se puede decir algo específico de las diferencias, sería que el modelo manual es ligeramente más conservativo para identificar a un churner. Esto podría ser un problema diminuto viéndolo en cuestiones prácticas, pero podría ser mejorado ligeramente si se reduce el threshold de probabilidad para ver quién se considera como churner y quién no, ya que actualmente todas las barreras están en un estándar de 0.5 de probabilidad.

Las causas de las diferencias se podrían atribuir a todos los métodos de optimización y la capacidad mucho mayor de llegar a una convergencia que contiene la librería de Scikit-learn, ya que esta librería tiene muchas más utilidades y sus hiperparámetros por defecto están seleccionados con mucha cautela para que puedan aplicarse de la mejor manera posible. Sin embargo, la implementación manual por descenso del gradiente demuestra que los fundamentos matemáticos pueden aplicarse correctamente para llegar a resultados similares a los estándar sin tener en cuenta todas las optimizaciones extra.

VI. CONCLUSIÓN

El desarrollo de este proyecto permitió validar de manera sólida que los fundamentos matemáticos del aprendizaje automático son suficientes para reproducir modelos ampliamente utilizados en la práctica, siempre que se implementen correctamente y se acompañen de un adecuado preprocesamiento de los datos. A través de la implementación manual de la regresión lineal y logística utilizando descenso del gradiente, fue posible no solo comprender a profundidad el comportamiento de estos algoritmos, sino también alcanzar resultados altamente comparables con los obtenidos mediante la librería *scikit-learn*.

En el caso de la regresión lineal, los experimentos evidenciaron que el algoritmo de descenso del gradiente converge hacia la solución óptima global cuando se seleccionan adecuadamente los hiperparámetros, particularmente la tasa de aprendizaje. Se observó que valores pequeños de α generan una convergencia lenta y soluciones subóptimas, mientras que valores mayores permiten alcanzar rápidamente el mínimo global, logrando replicar exactamente la solución analítica de *scikit-learn*. Este comportamiento confirma la naturaleza convexa del problema y resalta la importancia del ajuste de hiperparámetros en métodos iterativos.

Por otro lado, en la regresión logística, aunque la implementación manual no alcanzó una coincidencia exacta con la librería, sí logró resultados extremadamente cercanos en todas las métricas de evaluación. Las pequeñas diferencias observadas se atribuyen principalmente a las optimizaciones internas de *scikit-learn* y a la sensibilidad del modelo a parámetros como el número de iteraciones, la tasa de aprendizaje y el umbral de clasificación. Aun así, el modelo manual demostró ser capaz de construir fronteras de decisión muy similares, confirmando la correcta implementación del algoritmo.

En conjunto, los resultados obtenidos destacan que la clave del desempeño de estos modelos no reside únicamente en el uso de librerías avanzadas, sino en la comprensión profunda de sus principios matemáticos, el adecuado tratamiento de los datos y la correcta selección de hiperparámetros. Este trabajo no solo cumple con el objetivo de replicar modelos clásicos, sino que también refuerza la intuición sobre cómo y por qué funcionan, lo cual resulta esencial para abordar problemas más complejos en el campo del aprendizaje automático.

[1] M. I. Jordan and T. M. Mitchell, Machine learning: Trends, perspectives, and prospects, Science , 255 (2015).
[2] F. Provost and R. Kohavi, On applied research in machine learning, MACHINE LEARNING- BOSTON- , 127 (1998).
[3] S. Ruder, An overview of gradient descent optimization algorithms, (2016).
[4] P. Cunningham, M. Cord, and S. J. Delany, Supervised learning., In Machine learning techniques for multimedia: case studies on organization and retrieval , 21 (2008).
[5] S. H. Haji, A. M. Abdulazeez, *et al.*, Comparison of optimization techniques based on gradient descent algorithm: A review, PalArch's Journal of Archaeology of Egypt/Egyptology **18**, 2715 (2021).
[6] R. Cristin, P. Jeipratha, T. Daniya, and N. Kanagavalli, Logistic regression model with gradient descent parameter estimation, Journal of Critical Reviews **7**, 2048 (2020).